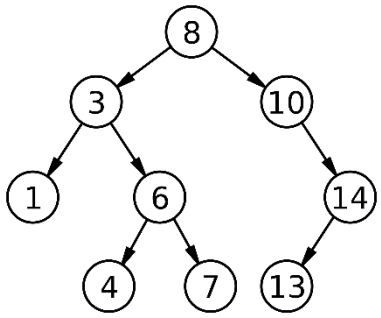
Università degli Studi di Trento Corso di Programmazione I – E3: 145430/145935 4° Appello – 03/07/2018 – a.a. 2017/2018	**PC N°**
COGNOME ..... **NOME .....** **MATRICOLA**	

Completare le voci **Cognome**, **Nome** e **Matricola**, se tali voci saranno mancanti o non-leggibili elaborato **NON** verrà corretto.
NON è consentito utilizzo di alcun strumento cartaceo, elettronico, informatico etc. durante la prova.
Fa parte della valutazione la leggibilità del codice C/C++.
Scrivere le risposte su questo foglio e riconsegnare. I risultati verranno pubblicati su DOL
Tempo a disposizione: 120 minuti.

PARTE TEORICA: 12 PUNTI – PUNTEGGIO MINIMO 6 PUNTI

[Domanda 1 – punti 3] Dato il seguente BST (vedi immagine) calcolare:

Cammino									
Altezza									
Visita pre-ordine									
Visita post-ordine									
Visita in-ordine									



[Domanda 2 - punti 1]
Di che cosa ho bisogno, come minimo, se devo codificare una grandezza che può assumere 734 valori distinti?

SI ☐ NO ☐ **1 byte**
SI ☐ NO ☐ **2 byte**
SI ☐ NO ☐ **4 byte**
SI ☐ NO ☐ **10 bit**
SI ☐ NO ☐ **1 kilobyte**
SI ☐ NO ☐ **mezzo byte**

[Domanda 3 - punti 4] Descrivere la differenza tra gestione memoria stack e heap: spiegare in dettaglio e dare degli esempi pratici.

[Domanda 4 - punti 4]

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 6
void mergesort (int a[], int left, int right){
    if (left < right) {
        int center = (left + right) / 2;
        printf("left=%d center=%d right=%d \n", left, center, right);
        mergesort(a, left, center);
        mergesort(a, center+1, right);
        merge(a, left, center, right); /*si suppone implementata*/
    }
}
void main(){
   //Inserisci le cifre della tua matricola, una alla volta
   int m[MAX];  int i;
   for (i= 0; i<MAX; i++)  { scanf("%d", &m[i]); }
   mergesort(m, 0, MAX-1);
}
```

Considerando l'esecuzione del programma su calcolatore, si scriva esattamente l'output mostrato a video.

MATRICOLA						

OUTPUT

PARTE PRATICA: 21 PUNTI – PUNTEGGIO MINIMO 11 PUNTI

[A - punti 6] *Inserire i seguenti tipi in un file* ***dati.h*** **e implementare i metodi di stampa ed i costruttori:**

```
typedef enum Testrazione { GAS, PETROLIO, ACQUA } Testrazione;
typedef struct Tpozzo{
  char zona[20];
  bool attivo;
  float quantita;
  Testrazione tipoEstrazione;
  Tpozzo () { /*  da implementare */ }
  void stampa() { /*  da implementare → formato: vedi punto E */ }
} Tpozzo;
typedef struct TstackLIFO {
   Tpozzo* s;
   int dim;
   int N;
   TstackLIFO () { /*  da implementare */ }
   TstackLIFO (int _dim) { /*  da implementare */ }
   bool stackIsFull () { /*  da implementare */ }
   bool stackIsEmpty () { /*  da implementare */ }
   void push(Tpozzo p) { /*  da implementare */ }
   Tpozzo pop () { /*  da implementare */ }
   void stampa() { /*  da implementare */ }
} TstackLIFO;
```

[B - punti 1] *Modificare il file* ***main.cpp*** *includendo il file* ***dati.h*** *ed inserendo il seguente codice nel* ***main****:*

```
TstackLIFO* pozzi[3];
for(int i=0; i<3; i++) { pozzi[i] = new TstackLIFO(5); }
Tpozzo p;
for(int i=0; i<5; i++) { creaPozzo(&p); addPozzo(pozzi,p); }
stampaPozzi(pozzi);
while (! pozzi[0]->stackIsEmpty() ) {
    p = pozzi[0]->pop();
    p.stampa();
}
```

[C - punti 4] Implementare una funzione **creaPozzo** che riceve in input per riferimento una variabile di tipo **Tpozzo p** e la inizializza con:

- campo **tipoEstrazione** valore casuale tra GAS, PETROLIO, ACQUA;
- campo **quantita** valore letto da tastiera, tra 100.00 e 800.00, con controllo input.
- campo **attivo** valore casuale tra true e false;
- campo **zona** letta da tastiera.

[D - punti 6] Implementare una funzione ***addPozzo*** che riceve come input una variabile di tipo array **TstackLIFO* pozzi** e variabile di tipo **Tpozzo s** e la inserisce nello stack **pozzi[x]** (con **x** valore casuale tra 0 e 2) (usare metodo **push**). Se lo stack risultasse pieno (metodo **stackIsFull)** aggiungere il dato ad un file denominato **"noprod.txt"** nella forma:

zona **QTA=**quantita **(**tipoEstrazione**)**

[E - punti 4] Implementare una funzione ***stampaPozzi*** che riceve in input una variabile di tipo array **TstackLIFO * pozzi** e stampa il contenuto delle liste utilizzando opportunamente il metodo **stampa di TstackLIFO**. Nella forma (per attivo stampare SI o NO):

zona **QTA=**quantita **(**tipoEstrazione**) attivo=**attivo

È possibile creare strutture, funzioni, metodi aggiuntivi per realizzare gli esercizi proposti.
Il codice commentato NON verrà corretto!